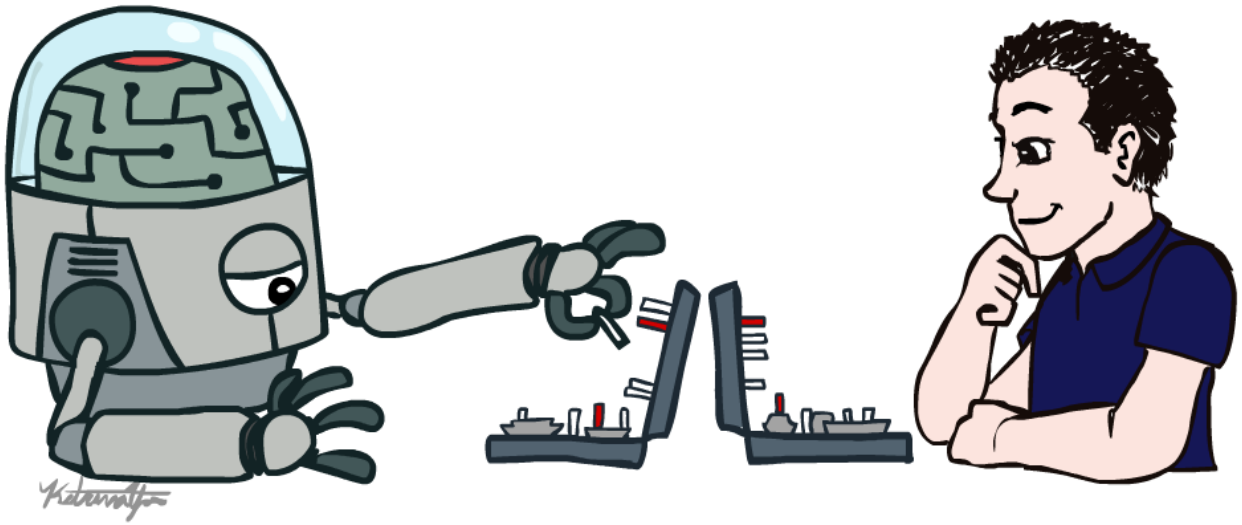
ICSI263: Машин сургалт

Density Estimation and Gaussian Mixture Models

Introducing Maximum Likelihood Estimation, and Gaussian Mixture Models



МУИС МТЭС ахлах багш Ph.D З.Цолмон

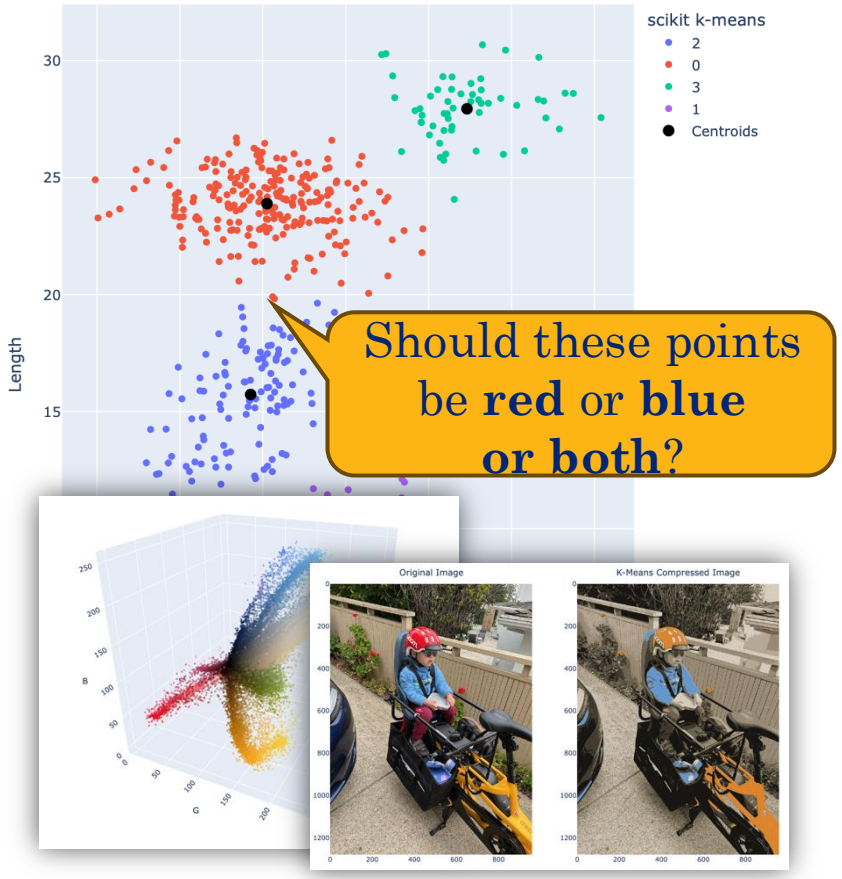
CC BY NC SA

In The Last Lecture We…

- Introduced **k-means clustering** as an **unsupervised clustering** algorithm.
  - Wanted a better way to capture uncertainty in cluster predictions.
- **Reviewed** basic ideas in probability.
- Explored **Bayesian updating** to determine the **false-positive rate** needed for a wake-word detector.







CC BY NC SA

Roadmap

- Expectation and Variance
- Density Estimation
  - Basic Probability Distributions
  - Maximum Likelihood
  - Bias
- Multivariate Normal
- Gaussian Mixture Models

Questions

Expectation and Variance

- Expectation and Variance
- Density Estimation
  - Basic Probability Distributions
  - Maximum Likelihood
  - Bias
- Multivariate Normal
- Gaussian Mixture Models

Questions

Expectations

The **expectation of a random variable** X is the the weighted average with respect to a probability distribution $p(x)$:

$$\mathbb{E}[X] = \sum_x x\, p(x)$$

Expectation is an **operator** from a **random variable** to a ***value***.

The **conditional expectation** of a random variable is a **weighted average** with respect to the **conditional distribution**:

$$\mathbb{E}[X \mid y] = \sum_x x\, p(x|y)$$

cc BY NC SA

Functions of Random Variables

The function of a random variable is also a random variable:

$$Y = f(X)$$

The **expectation of a function** $f(X)$ is the weighted average with respect to a probability distribution $p(x)$:

$$\mathbb{E}[f] = \mathbb{E}[f(X)] = \sum_x f(x)\, p(x)$$

The expectation of a function of multiple variables is defined over the joint distribution:

$$\mathbb{E}[f(X,Y)] = \sum_{x,y} f(x,y)\, p(x,y)$$

cc BY NC SA

Linearity of Expectation

For ***any*** two random variables X and Y:

$$\mathbb{E}[aX + bY + c] = a\mathbb{E}[X] + b\mathbb{E}[Y] + c$$

- Does not require any assumptions about $p(X, Y)$.

$$\begin{aligned}
\mathbb{E}[aX + bY + c] &= \sum_x \sum_y (a\,x + b\,y + c)\,p(x, y) \\
&= \sum_x \sum_y (a\,x\,p(x, y) + b\,y\,p(x, y) + c\,p(x, y)) \\
&= a \sum_x x \sum_y p(x, y) + b \sum_y y \sum_x p(x, y) + c \sum_x \sum_y p(x, y) \\
&= a\,\mathbb{E}[X] + b\mathbb{E}[Y] + c
\end{aligned}$$

cc BY NC SA

Variance

The **variance** of a random variable defines its **spread**:

$$\text{var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

Using linearity of expectation, we can rewrite the variance as:

$$\text{var}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

What is $\text{var}[a\,X + b]$?

$$\text{var}[a\,X + b] = \underbrace{\mathbb{E}[(a\,X + b)^2]} - \underbrace{\mathbb{E}[a\,X + b]^2}$$

$$\mathbb{E}[a^2X^2 + 2abX + b^2] \qquad - (a\mathbb{E}[X] + b)^2$$

$$a^2\mathbb{E}[X^2] + 2ab\mathbb{E}[X] + b^2 \qquad - a^2\mathbb{E}[X]^2 - 2ab\mathbb{E}[X] - b^2$$

$$\text{var}[a\,X + b] = a^2(\mathbb{E}[X^2] - \mathbb{E}[X]^2) = a^2\,\text{var}[X]$$

cc BY NC SA

Covariance

The covariance of two random variables defines the degree to which they vary together:

$$\text{cov}[X, Y] = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[X\,Y] - \mathbb{E}[X]\mathbb{E}[Y]$$

The covariance of two **random vectors** $\boldsymbol{x}$ and $\boldsymbol{y}$ is the matrix:

$$\text{cov}[\boldsymbol{x}, \boldsymbol{y}] = \mathbb{E}[(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])(\boldsymbol{y}^T - \mathbb{E}[\boldsymbol{y}^T])] = \mathbb{E}[\boldsymbol{x}\boldsymbol{y}^T] - \mathbb{E}[\boldsymbol{x}]\mathbb{E}[\boldsymbol{y}^T]$$

We will return to covariance in greater detail in future lectures.

cc BY NC SA

Probability Density Functions

For **continuous random variables**, we use **a density function $p(x)$** where the probability that x falls in the interval (a, b) is given by:

$$p\big(x \in (a,b)\big) = \int_a^b p(x)\mathrm{d}x$$

Just as with discrete distribution, we have the following two properties:

$$p(x) \geq 0 \quad \text{and} \quad \int_{-\infty}^{\infty} p(x)\mathrm{d}x = 1$$

We define the **cumulative distribution function**:

$$p(X \leq z) = P(z) = \int_{-\infty}^{z} p(x)\mathrm{d}x$$

The earlier discrete definitions apply but with integration replacing summation.

cc BY NC SA

Density Estimation

- Expectation and Variance
- Density Estimation
  - Basic Probability Distributions
  - Maximum Likelihood
  - Bias
- Multivariate Normal
- Gaussian Mixture Models

Questions

Density Estimation

Density estimation is the process of trying to **infer the probability distribution $\boldsymbol{p(X)}$** given observations (data) $\mathcal{D} = \{x_1, \ldots, x_N\}$.

There are *infinitely many* density functions from which our data could have been sampled.

- Could be any function that assigns **non-zero probability to the data.**
- The choice of function is a **modeling decision** (**inductive bias!**).

Density estimation is a fundamental technique in machine learning and is part of most modern methods.

CC BY NC SA

Estimating the Parameters of a Distribution

Given a collection of data $\mathcal{D} = \{x_1, \ldots, x_N\}$, there are several ways to estimate the "best" parameters for a given distribution.

- **Maximum Likelihood Estimation (MLE):** Choose the parameters that maximize the **likelihood** of the data. (Frequentist)

$$w^*_{\text{MLE}} = \arg\max_w p(\mathcal{D} \mid w)$$

- **Maximum A Posteriori (MAP) Estimation:** Choose the parameters that maximize the posterior given the data and the **prior**. (Bayesian)

$$w^*_{\text{MAP}} = \arg\max_w p(\mathcal{D} \mid w)\, p(w)$$

Both methods rely on computing the **likelihood of the data.**

- We will focus on MLE today.

cc BY NC SA

The Likelihood Function

The likelihood function describes the **probability of the observed data** $\mathcal{D} = \{x_1, \ldots, x_N\}$ under a particular distribution.

If we assume each observation $X_n \sim p(x|w)$ is an **Independent and Identically Distributed (IID)** sample from some parametric distribution $p(x|w)$ parameterized by w, then the likelihood is:

$$p(\mathcal{D}|w) = \prod_{n=1}^{N} p(x_n \mid w)$$

Independent

Identically

cc BY NC SA

The Log Likelihood Function

In practice, we will typically work with the **log likelihood function**:

$$p(\mathcal{D}|w) = \prod_{n=1}^{N} p(x_n \mid w) \quad \Longrightarrow \quad \ln p(\mathcal{D}|w) = \sum_{n=1}^{N} \ln p(x_n \mid w)$$

We take the **log** of the likelihood function for several reasons:

- It's **easier to manipulate summations** than products (e.g., calculus).
- The log probabilities and summation are **more numerically stable**.

Does this change our **optimization problem?**

- No – log is **monotonically increasing function** of its arguments:

$$\arg\max_{w} p(\mathcal{D}|w) = \arg\max_{w} \ln p(\mathcal{D}|w)$$

cc BY NC SA

Maximum Likelihood Estimation

Maximum likelihood estimation has two steps:

1. **Modeling:** Define the **log likelihood function** of the **data:** $\ln p(\mathcal{D}|w)$
2. **Optimization:** Compute the parameters that maximize the log-likelihood:

$$\theta_{\mathrm{ML}} = \arg\max_{\theta} \log p(\mathcal{D}|w)$$

Maximum likelihood estimation is a core technique in ML and is the basic objective for everything from the **logistic regression** to **ChatGPT.**

cc BY NC SA

Modeling: Bernoulli Distribution



The distribution of binary random variable $X \in \{0,1\}$ defined by:

$$X \sim \text{Bern}(x|\mu) \quad \Longrightarrow \quad p(x|\mu) = \mu^x (1-\mu)^{1-x}$$

where the parameter $0 \leq \mu \leq 1$.

$$\mathbb{E}[X] = \; 0\,(1-\mu) + 1\mu \; = \mu$$

$$\text{var}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2 \; = \mu - \mu^2 \; = \mu(1-\mu)$$

$$\mathbb{E}[X^2] = 0^2(1-\mu) + 1^2\mu = \mu$$

Note that for $\mu \notin \{0,1\}$, X never takes the expected value!

The Bernoulli distribution is used extensively in classification.

cc BY NC SA

The MLE for IID Bernoulli Samples (Part 1)

Example: Bernoulli Distribution

We observe N samples $\mathcal{D} = \{x_1, \ldots, x_N\}$ from a Bernoulli Distribution.

1. **Modeling:** Let's assume IID Bernoulli trials:

$$\ln p(\mathcal{D}|\mu) = \sum_{n=1}^{N} \ln\left(\mu^{x_n}(1-\mu)^{(1-x_n)}\right) = \sum_{n=1}^{N} x_n \ln(\mu) + (1-x_n)\ln(1-\mu)$$

Since each x_n is either **zero** or **one** we can rewrite this expression as:

$$\ln p(\mathcal{D}|\mu) = n_1 \ln(\mu) + n_0 \ln(1-\mu)$$

Where n_0 and n_1 are the number of observed zeros and ones, respectively.

- The counts n_0 and n_1 are called **sufficient statistics** because they **fully describe the data**.

cc BY NC SA

The MLE for IID Bernoulli Samples (Part 2)

2. Optimization: Maximize the log likelihood function:

$$\arg\max_{\mu} n_1 \ln(\mu) + n_0 \ln(1-\mu)$$

Here we use the **critical-points** approach to optimization:

1. Take the derivative $\frac{\partial}{\partial \mu} \ln p(\mathcal{D}|\mu)$:

$$\frac{\partial}{\partial \mu} \left(n_1 \ln(\mu) + n_0 \ln(1-\mu)\right) = \frac{n_1}{\mu} - \frac{n_0}{1-\mu}$$

2. Solve for the critical point $\frac{\partial}{\partial \mu} \ln p(\mathcal{D}|\mu) = 0$

$$\frac{n_1}{\mu} - \frac{n_0}{1-\mu} = 0 \implies n_1(1-\mu) = n_0\mu \implies \mu_{ML} = \frac{n_1}{n_0 + n_1}$$

3. Verify that critical points are maxima $\frac{\partial^2}{\partial \mu^2} \ln p(\mathcal{D}|\mu) < 0$

cc BY NC SA

The MLE for ChatGPT

We model the likelihood of all the documents on the internet:

$$\ln p(\text{Internet} \mid w) = \sum_{\text{Doc} \in \text{Internet}} \sum_{\text{Word} \in \text{Doc}} \ln p(\text{Word} \mid \text{Words before it in Doc}, w)$$

Pre-Training: billions of dollars spent annually computing an MLE:

$$\arg\max_{w} \ln p(\text{Interent} \mid w)$$

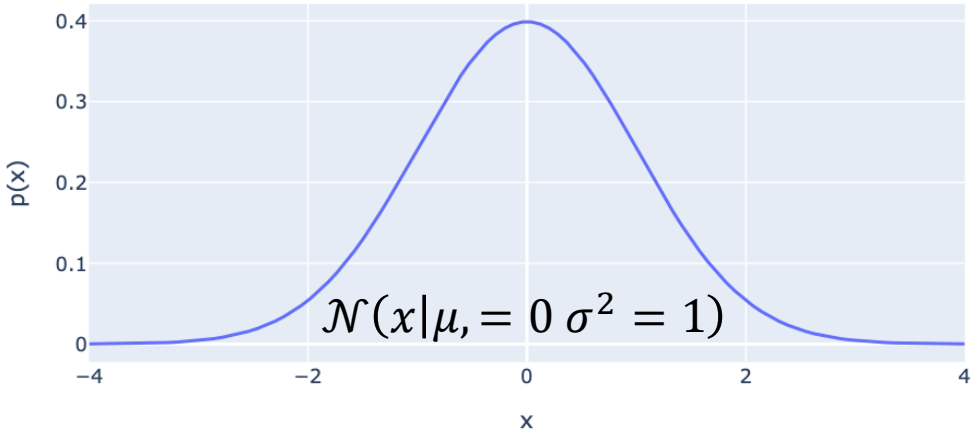
cc BY NC SA

Normal (Gaussian) Distribution

If a single real-valued variable X is **normally distributed,** then:

$$X \sim \mathcal{N}(x|\mu, \sigma^2) \quad \Rightarrow \quad p(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\,\pi\,\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right)$$

Where the parameters μ and σ^2 are called the mean and variance.



With some calculus you can show $\mathbb{E}[X] = \mu$ and $\text{var}[X] = \sigma^2$.

The MLE for IID Gaussian Samples (Part 1)

Example: Gaussian Distribution

We observe N continuous real valued samples $\mathcal{D} = \{x_1, \ldots, x_n\}$.

1. **Modeling:** Let's assume IID Gaussian trials:

$$\ln p(\mathcal{D}|\mu, \sigma^2) = \sum_{n=1}^{N} \ln\left(\frac{1}{\sqrt{2\,\pi\,\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x_n - \mu)^2\right)\right)$$

$$= -\frac{N}{2}\ln(2\,\pi) - \frac{N}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n - \mu)^2$$

2. **Optimization:** Maximize the log likelihood function:

$$\arg\max_{\mu,\sigma^2} -\frac{N}{2}\ln(2\,\pi) - \frac{N}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n - \mu)^2$$

cc BY NC SA

The MLE for IID Gaussian Samples (Part 2)

Maximize the log likelihood function:

$$\arg\max_{\mu,\sigma^2} -\frac{N}{2}\ln(2\,\pi) - \frac{N}{2}\ln(\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n - \mu)^2$$

Compute derivatives with respect to μ and σ^2 and solve for the critical-points.

$$\frac{\partial}{\partial\mu}\ln p(\mathcal{D}|\mu,\sigma^2) = \frac{1}{\sigma^2}\sum_{n=1}^{N}(x_n - \mu)$$

Solving for μ_{ML} :

$$\frac{1}{\sigma^2}\sum_{n=1}^{N}(x_n - \mu) = 0 \implies \mu_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}x_n$$

cc BY NC SA

The MLE for IID Gaussian Samples (Part 2)

Computing the derivative with respect to the variance parameter $v = \sigma^2$

$$\frac{\partial}{\partial v}\ln p(\mathcal{D}|\mu, v) = \frac{\partial}{\partial v}\left(-\frac{N}{2}\ln(v) - \frac{1}{2v}\sum_{n=1}^{N}(x_n - \mu)^2\right) = -\frac{N}{2}\left(\frac{1}{v}\right) + \frac{1}{2v^2}\sum_{n=1}^{N}(x_n - \mu_{\mathrm{ML}})^2$$

Set equal to zero and solving for $v_{\mathrm{ML}} = \sigma^2_{\mathrm{ML}}$:

$$-\frac{N}{2}\left(\frac{1}{v}\right) + \frac{1}{2v^2}\sum_{n=1}^{N}(x_n - \mu_{\mathrm{ML}})^2 = 0$$

$$\frac{v^2}{N} * \quad N\left(\frac{1}{v}\right) = \frac{1}{v^2}\sum_{n=1}^{N}(x_n - \mu_{\mathrm{ML}})^2 \quad * \frac{v^2}{N}$$

$$v_{\mathrm{ML}} = \sigma^2_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}(x_n - \mu_{\mathrm{ML}})^2$$

cc BY NC SA

Recap of the MLE for IID Gaussian Samples

We used the method of **Maximum Likelihood** to estimate the mean and variance parameters of a Gaussian from data:

$$\mu_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N} x_n$$

Sufficient Statistics

$$\sigma^2_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}(x_n - \mu_{\mathrm{ML}})^2 = \frac{1}{N}\left(\sum_{n=1}^{N} x_n^2\right) - \mu^2_{\mathrm{ML}}$$

What are the sufficient statistics?

Do they match our intuition for the mean and variance.

Are they good estimates?

cc BY NC SA

Bias

- Expectation and Variance
- Density Estimation
  - Basic Probability Distributions
  - Maximum Likelihood
  - Bias
- Multivariate Normal
- Gaussian Mixture Models

Questions

Bias of an Estimate

How close is the true (unknown) parameter value to our estimate in expectation?

Before we observe our data, the value of our estimate θ_{ML} is a **random variable**.

We can compute the bias of the estimation procedure as:

$$\mathrm{Bias}(\theta_{\mathrm{ML}}) = \mathbb{E}[\theta_{\mathrm{ML}}] - \theta$$

Relative to the true (but unknown) θ.

cc BY NC SA

Bias of the MLE for the Gaussian Distribution

Assuming the data are drawn IID from $\mathcal{N}(x|\mu,\sigma^2)$ treating X_n, μ_{ML}, and σ^2_{ML} as random variables

$$\mathbb{E}[\mu_{\mathrm{ML}}] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[X_n] = \frac{1}{N}\sum_{n=1}^{N}\mu = \boxed{\mu}$$ **Unbiased**

Now looking at the variance:

$$\mathbb{E}[\sigma^2_{\mathrm{ML}}] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[X_n^2] - \mathbb{E}[\mu^2_{\mathrm{ML}}]$$

$$= \frac{1}{N}\sum_{n=1}^{N}(\sigma^2+\mu^2) - (\mathrm{var}(\mu_{\mathrm{ML}}) + \mathbb{E}[\mu_{\mathrm{ML}}]^2)$$

Variance Identity

$$\mathrm{var}(X_n) = \mathbb{E}[X_n^2] - \mu^2 = \sigma^2$$
$$\mathrm{var}(\mu_{\mathrm{ML}}) = \mathbb{E}[\mu^2_{\mathrm{ML}}] - \mathbb{E}[\mu_{\mathrm{ML}}]^2$$

$$\mathrm{var}(\mu_{\mathrm{ML}}) = \mathrm{var}\left(\frac{1}{N}\sum_{n=1}^{N}X_n\right) = \frac{1}{N^2}\left(\sum_{n=1}^{N}\mathrm{var}(X_n)\right) = \frac{1}{N^2}N\underbrace{\sigma^2}_{\mathrm{var}(X_n)} = \frac{\sigma^2}{N}$$

Independence of X_n

cc BY NC SA

Bias of the MLE for the Gaussian Distribution

Assuming the data are drawn IID from $\mathcal{N}(x|\mu,\sigma^2)$ treating X_n, μ_{ML}, and σ^2_{ML} as random variables

$$\mathbb{E}[\mu_{\mathrm{ML}}] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[X_n] = \frac{1}{N}\sum_{n=1}^{N}\mu = \boxed{\mu}$$ **Unbiased**

Now looking at the variance:

$$\mathbb{E}[\sigma^2_{\mathrm{ML}}] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[X_n^2] - \mathbb{E}[\mu^2_{\mathrm{ML}}]$$

$$= \frac{1}{N}\sum_{n=1}^{N}(\sigma^2+\mu^2) - (\mathrm{var}(\mu_{\mathrm{ML}}) + \mathbb{E}[\mu_{\mathrm{ML}}]^2)$$

$$= (\sigma^2+\mu^2) - \left(\frac{\sigma^2}{N}+\mu^2\right) = \boxed{\sigma^2\left(\frac{N-1}{N}\right)}$$

Biased

Variance Identity

$$\mathrm{var}(X_n) = \mathbb{E}[X_n^2] - \mu^2 = \sigma^2$$
$$\mathrm{var}(\mu_{\mathrm{ML}}) = \mathbb{E}[\mu^2_{\mathrm{ML}}] - \mathbb{E}[\mu_{\mathrm{ML}}]^2$$

Sample Mean Variance

$$\mathrm{var}(\mu_{\mathrm{ML}}) = \frac{\sigma^2}{N}$$

cc BY NC SA

Multivariate Normal

- Expectation and Variance
- Density Estimation
  - Basic Probability Distributions
  - Maximum Likelihood
  - Bias
- Multivariate Normal
- Gaussian Mixture Models

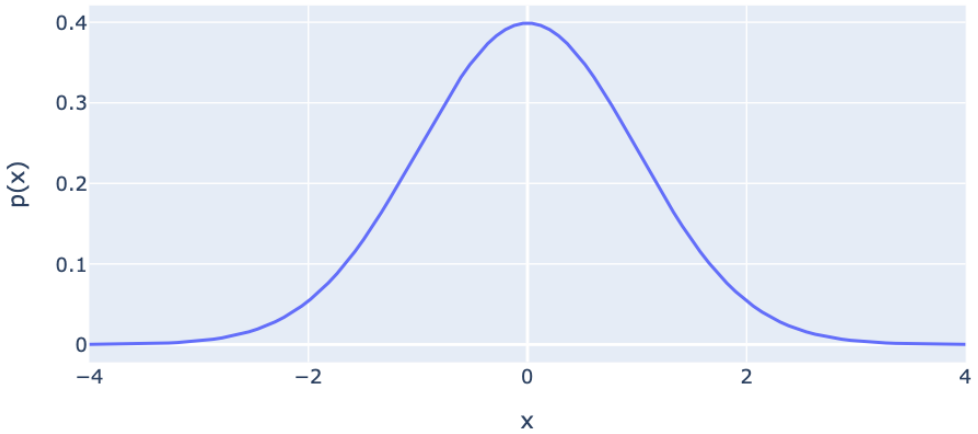
Questions

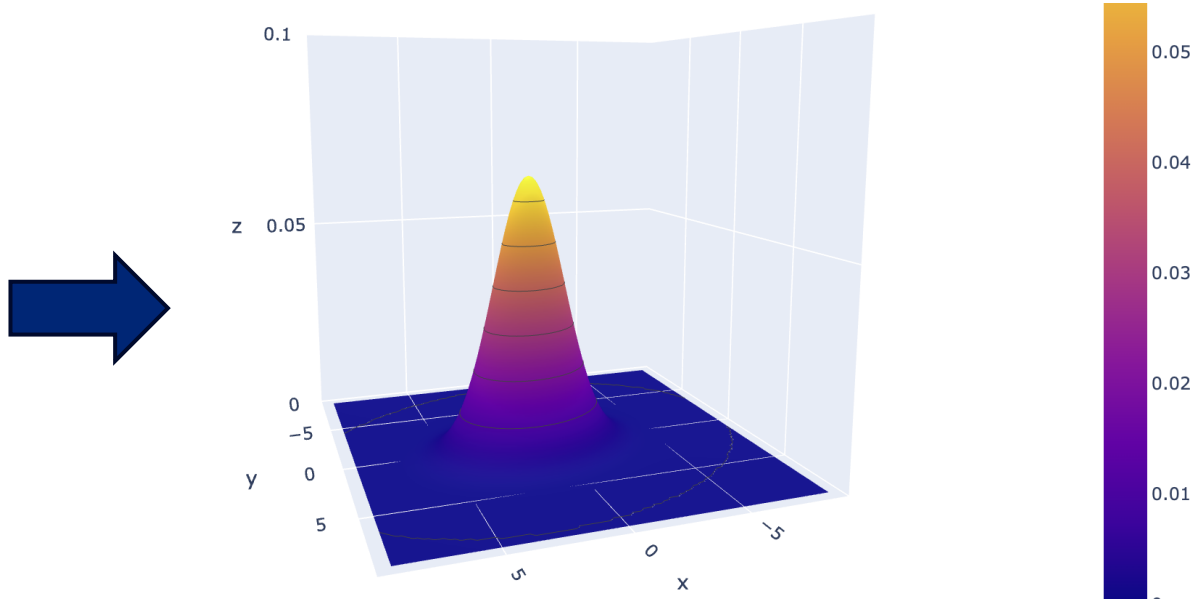
Multivariate Gaussian Distribution

The **probability density function** for a multivariate Gaussian distribution in D dimensions is give by:

$$\mathcal{N}(\mathbf{x}\,|\mu,\Sigma) = \frac{1}{(2\pi)^{D/2}}\frac{1}{|\Sigma|^{1/2}}\exp\left(-\frac{1}{2}(\mathbf{x}-\mu)^T\,\Sigma^{-1}(\mathbf{x}-\mu)\right)$$

where the **parameters** $\mu \in \mathbb{R}^D$ and $\Sigma \in \mathbb{R}^{DxD}$ are the **mean vector** and **covariance matrix**.





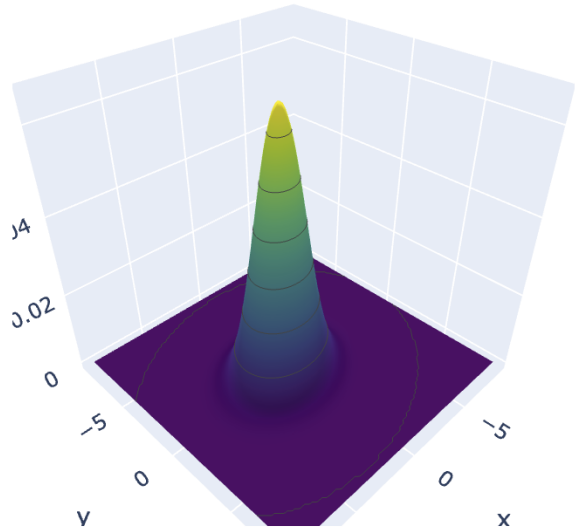
CC BY NC SA

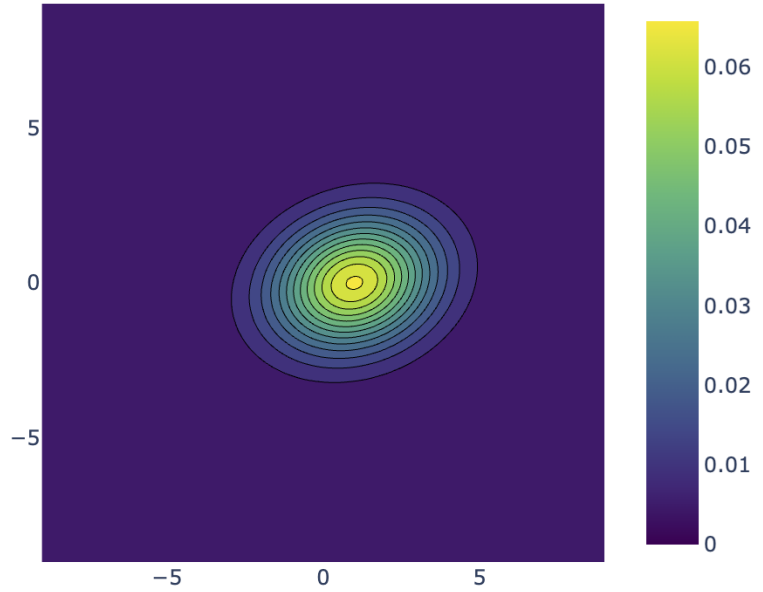
Demo

Gaussians

```
mu = np.array([1, 0])
Sigma = np.array([[3, 0.4], [0.4, 2]])

from scipy.stats import multivariate_normal
normal = multivariate_normal(mean=mu, cov=Sigma)
normal.pdf(np.array([[1, 0.5]]))
```





Gaussian Mixture Models

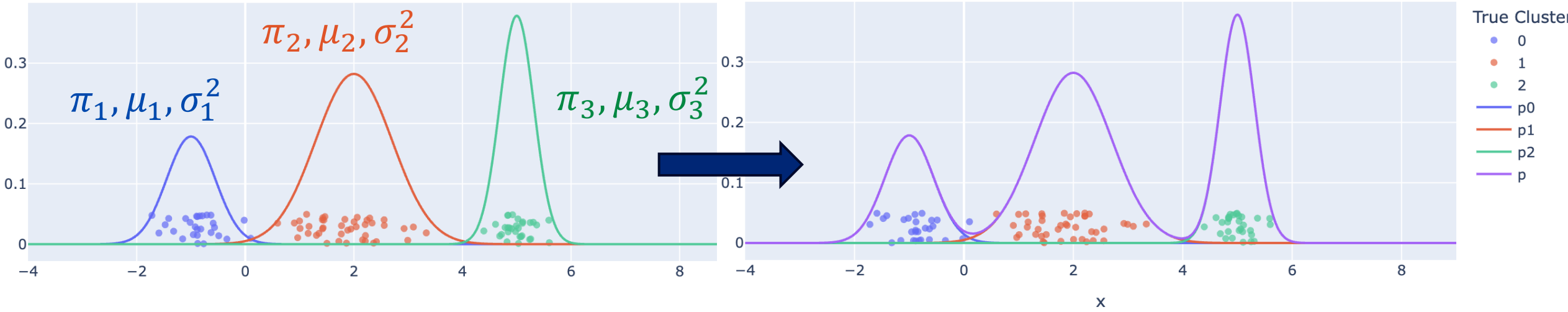
- Expectation and Variance
- Density Estimation
  - Basic Probability Distributions
  - Maximum Likelihood
  - Bias
- Multivariate Normal
- Gaussian Mixture Models

Questions

Gaussian Mixture Model (GMM)

The Gaussian mixture model defines the probability of sampling a D-dimensional vector **x** as a **weighted combination** of Gaussians:

- **Example:** K=3



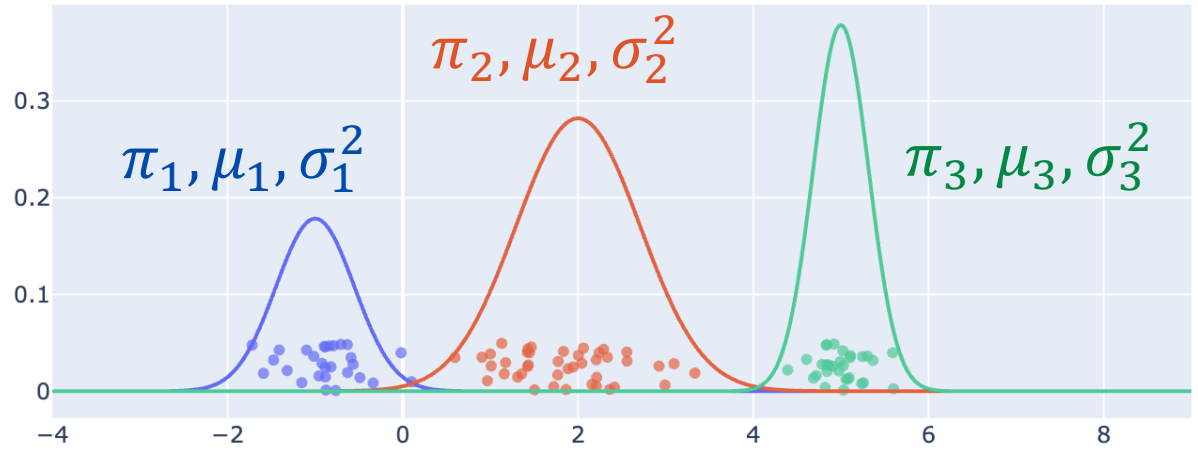
Gaussian Mixture Model (GMM)

The Gaussian mixture model defines the probability of sampling a D-dimensional vector $\mathbf{x}$ as a **weighted combination** of Gaussians:

$$p(\mathbf{x} \mid \pi, \mu_1, \Sigma_1, \ldots, \mu_K, \Sigma_K) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$

where $\sum_{k=1}^{K} \pi_k = 1$ is a probability distribution over clusters.

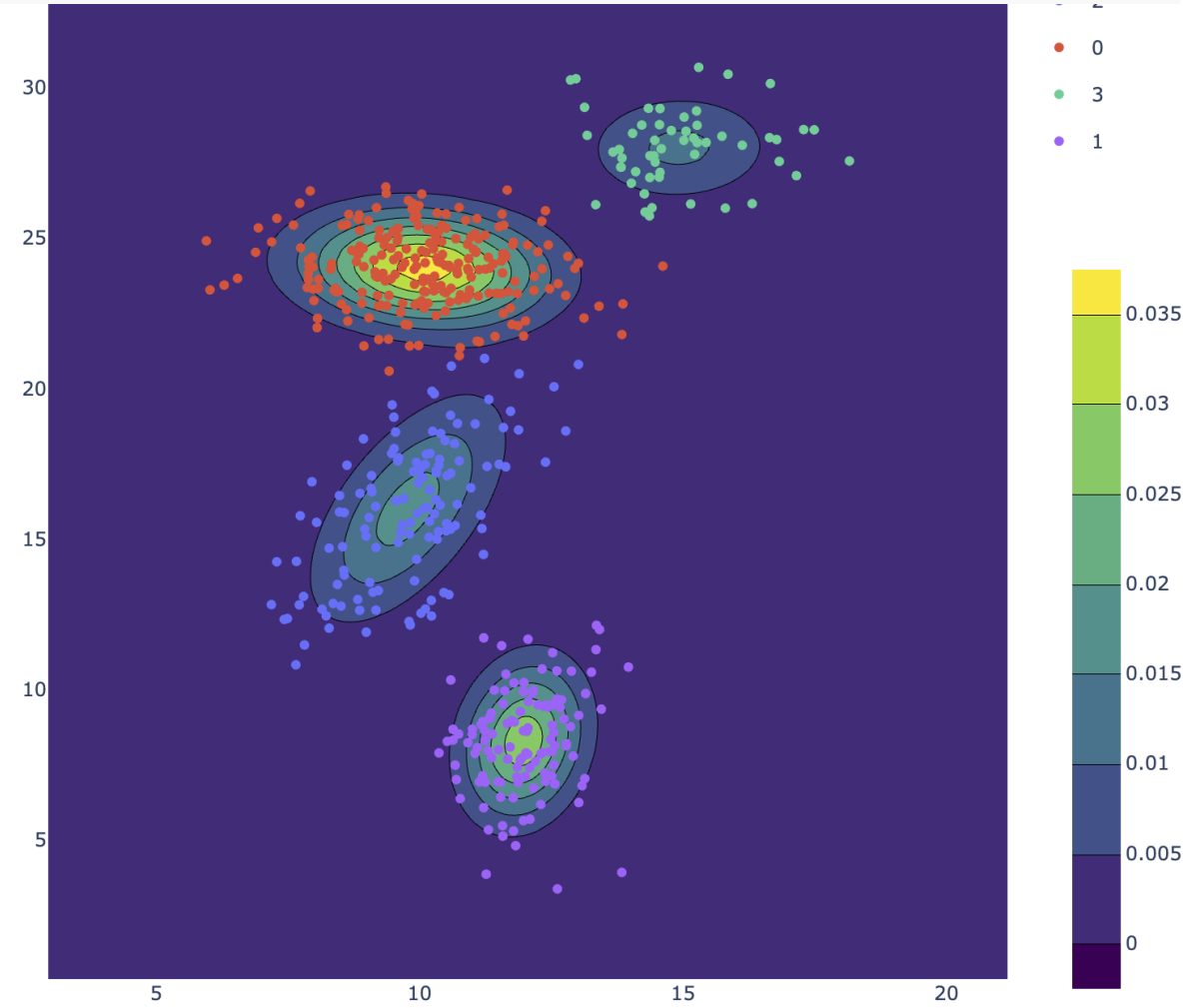
- **Example:** $K=3$



Demo

Gaussian Mixture Model

```
from sklearn.mixture import GaussianMixture
# Create a Gaussian Mixture Model with 4 components
gmm = GaussianMixture(n_components=4, random_state=42, )
# Fit the model to the data
gmm.fit(bikes[['Speed', 'Length']])
# Get the cluster labels
bikes['scikit gmm'] = gmm.predict(bikes[['Speed', 'Length']]).astype(str)
```



The GMM is a **Latent Variable** Model

We introduce a **latent** (unobserved) **assignment variable** z for each $\mathbf{x}$.

$$p(z) = \pi_z$$

We then define the likelihood of $\mathbf{x}$ given z as:

$$p(\mathbf{x} \mid z) = \mathcal{N}(\mathbf{x} \mid \mu_z, \Sigma_z)$$

The joint probability of the $\mathbf{x}$ and z is then:

$$p(\mathbf{x}, z) = p(\mathbf{x}| z)p(z) = \mathcal{N}(\mathbf{x} \mid \mu_z, \Sigma_z)\pi_z$$

If we **marginalize over z** we obtain the **GMM**:

$$p(\mathbf{x}) = \sum_{k=1}^{K} p(\mathbf{x}, z = k) = \sum_{k=1}^{K} \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)\pi_k$$

cc BY NC SA

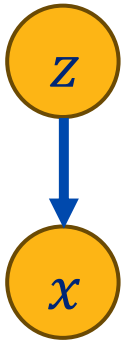
The GMM is a **Generative Model**

A **generative model** is a model that can **generate the data**.

- The GMM is a generative model.

Recall, the joint distribution has the following factorization:

$$p(\mathbf{x}, z) = p(z)\, p(\mathbf{x} \mid z)$$



We can draw a **graph** where the parents of a variable are the variables in its **conditional distribution.**

- This is a **graphical model** and is used to represent the **conditional structure** of a probability model.

We can sample data from a graphical model using **ancestor sampling.**

- Sample all the variables that have no parents: $p(z)$
- Continue sampling variables after all their parents have been sampled: $p(\mathbf{x} \mid z)$

CC BY NC SA

Demo

Sampling from a GMM

```
# Ancestor Sampling to create a synthetic dataset
np.random.seed(42)
N = 100

mu = np.array([-1, 2, 5])
pi = np.array([0.2, 0.5, 0.3])
Sigma = np.array([0.2, 0.5, .1])
```

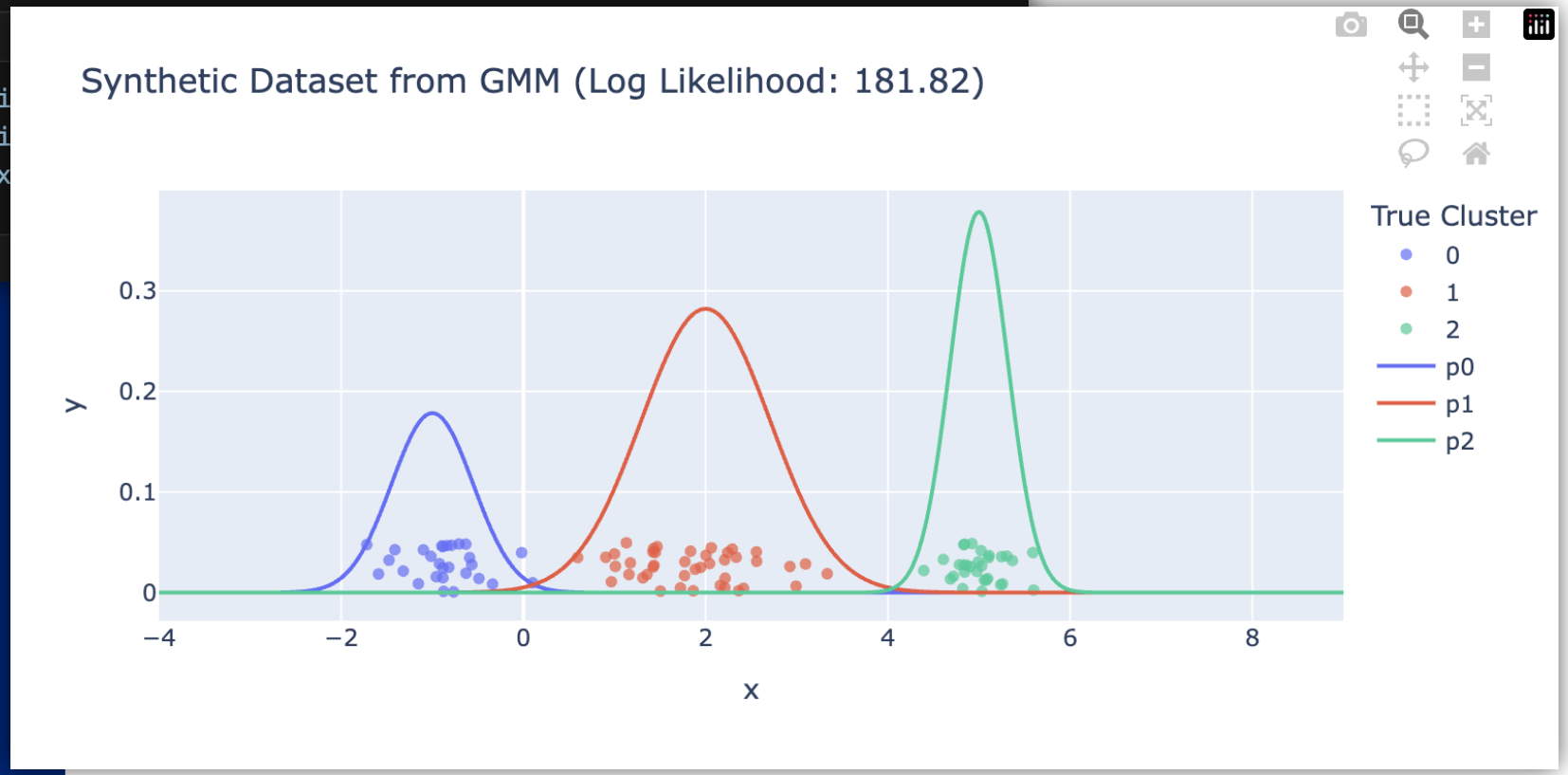
✓ 0.0s

```
z = np.random.choice(len(mu), size=N, p=pi)
x = np.random.normal(mu[z], np.sqrt(Sigma[z]))
```

✓ 0.0s

```
log_li
    pi
    ax
```

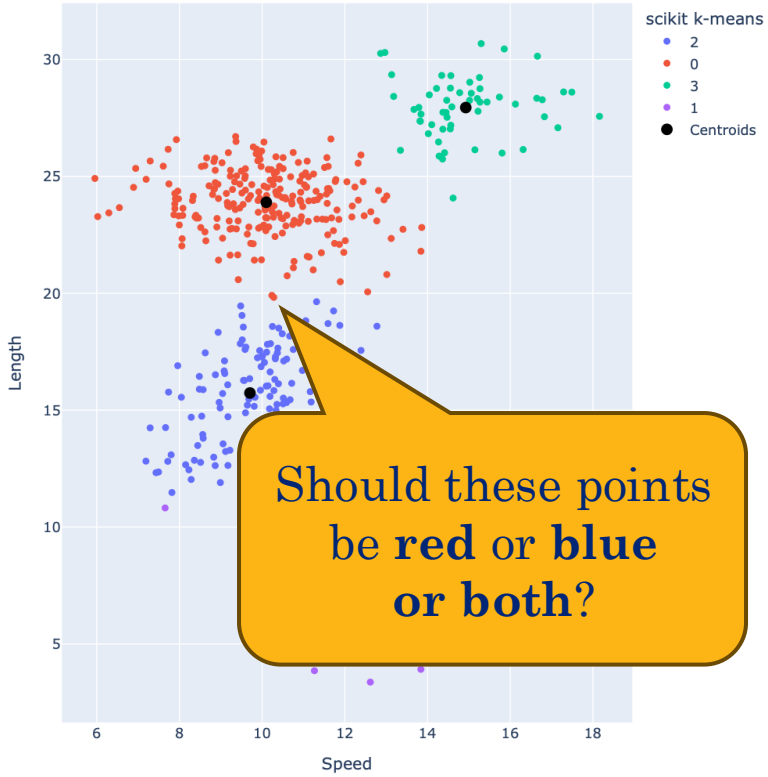
✓ 0.0s



Latent Variable Posteriors

We are interested in modeling the **distribution** (*the uncertainty*) over the **cluster assignment** z.

- **Prior:** $p(z) = \pi_z$
- **Likelihood:** $p(\mathbf{x} \mid z) = \mathcal{N}(\mathbf{x} \mid \mu_z, \Sigma_z)$



We can use Bayes rule to compute the posterior:

$$p\left(z \mid \mathbf{x}, \pi, \{\mu_k, \Sigma_k\}_{k=1}^K\right) = \frac{p(\mathbf{x} \mid z)p(z)}{\sum_{k=1}^K p(\mathbf{x} \mid z = k)p(z = k)} = \frac{\mathcal{N}(\mathbf{x} \mid \mu_z, \Sigma_z)\pi_z}{\sum_{k=1}^K \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)\pi_k}$$

If we **could estimate** $\pi, \{\mu_k, \Sigma_k\}_{k=1}^K$ **from the data** we could compute this distribution over cluster assignments.

cc BY NC SA

Estimating the GMM Parameters

We could try to use the method of **maximum likelihood:**

$$\ln\left(p\left(\{\mathbf{x}_n\}_{n=1}^N \mid \pi, \{\mu_k, \Sigma_k\}_{k=1}^K\right)\right) = \sum_{n=1}^{N} \ln\left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}_n \mid \mu_k, \Sigma_k)\right)$$

- however, the summation inside of the log couples the parameters.

But, if we extend the model to include the **latent variable** z

$$\ln\left(p\left(\{\mathbf{x}_n, z_n\}_{n=1}^N \mid \pi, \{\mu_k, \Sigma_k\}_{k=1}^K\right)\right) = \sum_{n=1}^{N} \left(\ln(\pi_{z_n}) + \ln\left(\mathcal{N}\left(\mathbf{x}_n \mid \mu_{z_n}, \Sigma_{z_n}\right)\right)\right)$$

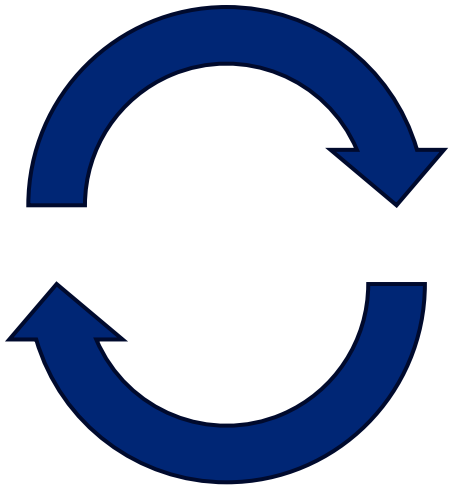
- We can optimize the parameters directly (sum of log terms).
- But we don't know the values of $\{z_n\}_{n=1}^N$

CC BY NC SA

Quick Recap

If we knew the model parameters, we could easily compute the cluster assignments.

Model Parameters $\pi, \{\mu_k, \Sigma_k\}_{k=1}^{K}$



Cluster Assignments $\{z_n\}_{n=1}^{N}$

If we knew the cluster assignments, we could easily estimate the model parameters.

How can we solve this cyclic dependency?

cc BY NC SA

The EM Algorithm

The **Expectation-Maximization (EM)** algorithm is a standard technique for fitting **latent variable** models like the Gaussian Mixture Model.

1. **Initialize** the model parameters θ.
2. **Iterate** until convergence.
   - **E-Step:** Compute expected log-likelihood.

$$Q(\theta') = \sum_{Z} \ln(p(X, Z \mid \theta'))\, p(Z \mid X, \theta)$$

Easy to optimize **joint probability**

Current distribution over the latent Z

   - **M-Step:** Maximize the expected log-likelihood.

$$\theta = \arg\max_{\theta'} Q(\theta')$$

Updates distribution over Z.

CC BY NC SA

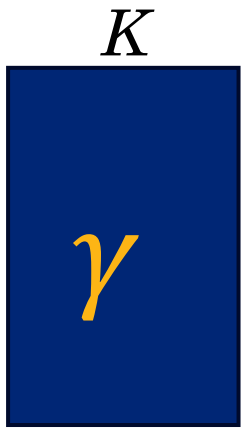
The EM Algorithm: **E-step**

E-Step: We don't know the value of $\{z_n\}_{n=1}^{N}$ but we can compute

$$\gamma_{nk} = p\left(z_n = k \,|\mathbf{x}_n, \pi, \{\mu_k, \Sigma_k\}_{k=1}^{K}\right) = \frac{p(\mathbf{x}_n \mid z_n)p(z_n)}{\sum_{k'=1}^{K} p(\mathbf{x}_n|k')p(k')} = \frac{\mathcal{N}\left(\mathbf{x}_n|\, \mu_{z_n}, \Sigma_{z_n}\right)\pi_{z_n}}{\sum_{k'=1}^{K} \mathcal{N}(\mathbf{x}_n|\, \mu_{k'}, \Sigma_{k'})\pi_{k'}}$$

- γ_{nk} are "soft" assignments to each class.

Using the γ_{nk} we can compute the **expected log likelihood:**



$$Q\left(\pi, \{\mu_k, \Sigma_k\}_{k=1}^{K}\right) = \sum_{n=1}^{N}\sum_{k=1}^{K} \gamma_{nk} \ln\left(p\left(\mathbf{x}_n, z_n = k \mid \pi, \{\mu_k, \Sigma_k\}_{k=1}^{K}\right)\right)$$

$$= \sum_{n=1}^{N}\sum_{k=1}^{K} \gamma_{nk}\left(\ln(\pi_k) + \ln\left(\mathcal{N}(\mathbf{x}_n|\, \mu_k, \Sigma_k)\right)\right)$$

cc BY NC SA

The EM Algorithm: **M-Step** (π_k)

M-Step: Compute the parameters that maximize Q:

$$Q\left(\pi, \{\mu_k, \Sigma_k\}_{k=1}^{K}\right) = \sum_{n=1}^{N}\sum_{k=1}^{K} \gamma_{nk}\left(\ln(\pi_k) + \ln\left(\mathcal{N}(\mathbf{x}_n|\,\mu_k, \Sigma_k)\right)\right)$$

Optimizing with respect to π_k:

Lagrangian for the **normalization constraint**

$$\frac{\partial}{\partial \pi_k}\sum_{n=1}^{N}\sum_{k=1}^{K} \gamma_{nk}\left(\ln(\pi_k) + \ln\left(\mathcal{N}(\mathbf{x}_n|\,\mu_k, \Sigma_k)\right)\right) + \lambda\left(\sum_{k=1}^{K}\pi_k - 1\right) = \sum_{n=1}^{N}\frac{\gamma_{nk}}{\pi_k} + \lambda = 0$$

$$1 = \sum_{k=1}^{K}\pi_k = -\frac{1}{\lambda}\sum_{n=1}^{N}\sum_{k=1}^{K}\gamma_{nk} = -\frac{1}{\lambda}\sum_{n=1}^{N}1$$
$$= -\frac{N}{\lambda} \Rightarrow \boldsymbol{\lambda = -N}$$

$$\pi_k = -\frac{1}{\lambda}\sum_{n=1}^{N}\gamma_{nk}$$
$$= \frac{1}{N}\sum_{n=1}^{N}\gamma_{nk} = \frac{N_k}{N}$$

$$N_k = \sum_{n=1}^{N}\gamma_{nk}$$

CC BY NC SA

The EM Algorithm: **M-Step** (μ_k)

M-Step: Compute the parameters that maximize Q:

$$Q\left(\pi, \{\mu_k, \Sigma_k\}_{k=1}^{K}\right) = \sum_{n=1}^{N}\sum_{k=1}^{K} \gamma_{nk}\left(\ln(\pi_k) + \ln\left(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)\right)\right)$$

Optimizing with respect to μ_k:

$$\frac{\partial}{\partial \mu_k}\sum_{n=1}^{N}\sum_{k=1}^{K} \gamma_{nk}\left(\ln(\pi_k) + \ln\left(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)\right)\right) = \sum_{n=1}^{N} \gamma_{nk}\, \Sigma_k^{-1}(\mathbf{x}_n - \mu_k) = 0$$

$$\mu_k = \frac{1}{\sum_{n=1}^{N} \gamma_{nk}} \sum_{n=1}^{N} \gamma_{nk}\, \mathbf{x}_n = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_{nk}\, \mathbf{x}_n$$

CC BY NC SA

The EM Algorithm: **M-Step** (Σ_k)

M-Step: Compute the parameters that maximize Q:

$$Q\left(\pi, \{\mu_k, \Sigma_k\}_{k=1}^{K}\right) = \sum_{n=1}^{N}\sum_{k=1}^{K} \gamma_{nk}\left(\ln(\pi_k) + \ln\left(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)\right)\right)$$

Optimizing with respect to Σ_k requires matrix calculus (out of scope):

$$\Sigma_k = \frac{1}{N_k}\sum_{n=1}^{N} \gamma_{nk}\,(\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^T$$

Summarizing the other parameters:

$$N_k = \sum_{n=1}^{N} \gamma_{nk}\,, \qquad \pi_k = \frac{N_k}{N}\,, \qquad \mu_k = \frac{1}{N_k}\sum_{n=1}^{N} \gamma_{nk}\,\mathbf{x}_n$$

CC BY NC SA

The EM Algorithm for GMMs

1. **Initialize** the model parameters $\pi, \{\mu_k, \Sigma_k\}_{k=1}^K$ (often using k-means)
2. **Iterate** until convergence
   - **E-Step:** compute expected log-likelihood

$$\gamma_{nk} = \frac{\mathcal{N}\left(\mathbf{x}_n \middle| \mu_{z_n}, \Sigma_{z_n}\right)\pi_{z_n}}{\sum_{k'=1}^{K} \mathcal{N}(\mathbf{x}_n | \mu_{k'}, \Sigma_{k'})\pi_{k'}}$$

   - **M-Step:** maximize the expected log-likelihood

$$N_k = \sum_{n=1}^{N} \gamma_{nk}\,, \qquad \pi_k = \frac{N_k}{N}\,, \qquad \mu_k = \frac{1}{N_k}\sum_{n=1}^{N} \gamma_{nk}\,\mathbf{x}_n$$

$$\Sigma_k = \frac{1}{N_k}\sum_{n=1}^{N} \gamma_{nk}\,(\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^T$$

CC BY NC SA

Demo

Implement GMMs

```
from sklearn.mixture import GaussianMixture
# Create a Gaussian Mixture Model with 4 components
gmm = GaussianMixture(n_components=4, random_state=42, )
# Fit the model to the data
gmm.fit(bikes[['Speed', 'Length']])
# Get the cluster labels
bikes['scikit gmm'] = gmm.predict(bikes[['Speed', 'Length']]).astype(str)
```

